GLOBALRAIN

Practices for Secure Software Report

Table of Contents

Document Revision History

Version	Date	Author	Comments
1.0	06/19/26	Miranda Perez	

Client

Instructions

Submit this completed practices for secure software report. Replace the bracketed text with the relevant information. You must document your process for writing secure communications and refactoring code that complies with software security testing protocols.

- Respond to the steps outlined below and include your findings.
- Respond using your own words. You may also choose to include images or supporting materials. If you include them, make certain to insert them in all the relevant locations in the document.
- Refer to the Project Two Guidelines and Rubric for more detailed instructions about each section of the template.

Developer
Miranda Perez

1. Algorithm Cipher

For this project, the SHA-256 cryptographic hash algorithm was selected to provide data verification and integrity. SHA-256 is part of the Secure Hash Algorithm (SHA-2) family developed by the National Security Agency (NSA). The algorithm generates a fixed-length 256-bit hash value from an input string, allowing users to verify that data has not been altered during transmission. SHA-256 is a one-way hash function that produces a unique 256-bit checksum for input data. Even a small change to the original data produces a completely different hash value. Because hash functions are one-way operations, the original data cannot be reconstructed from the hash, making SHA-256 ideal for verifying data integrity.

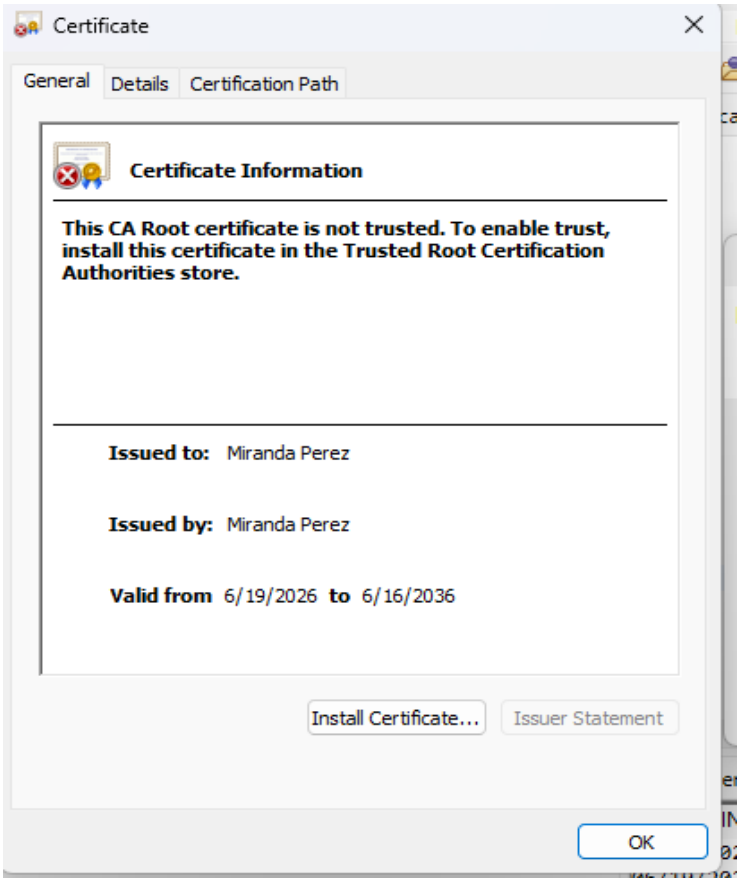
Unlike encryption algorithms, SHA-256 does not use symmetric or asymmetric keys because it is a hashing algorithm rather than an encryption cipher. Symmetric encryption uses a single shared key for encryption and decryption, while asymmetric encryption uses public and private key pairs. Random numbers are commonly used in encryption systems to generate secure keys, initialization vectors, and certificates. In this project, a self-signed certificate using RSA public-key cryptography was implemented to secure communications through HTTPS.

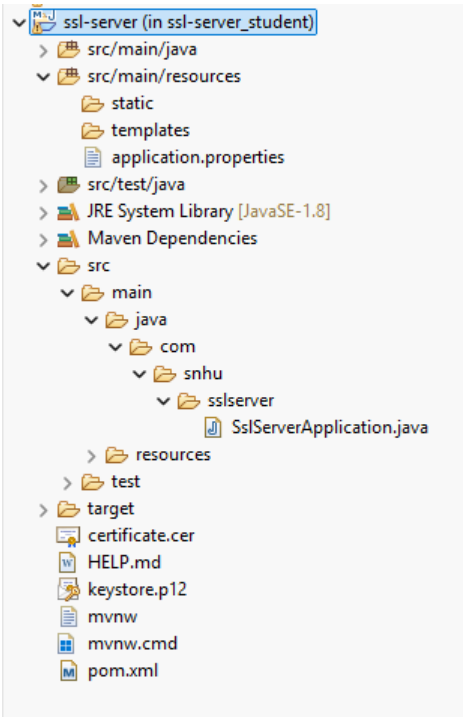
Early encryption algorithms such as DES have become obsolete due to advances in computing power. Modern standards such as AES and RSA are widely used for secure communication and data protection. SHA-256 remains a current industry-standard hashing algorithm because of its strong resistance to collisions and preimage attacks. As a result, SHA-256 is commonly used in digital signatures, SSL/TLS certificates, and blockchain technologies.

2. Certificate Generation

Insert a screenshot below of the CER file.

A self-signed SSL certificate was generated using the Java Keytool utility and exported as a CER file. The certificate was configured for the Artemis Financial application and used to enable HTTPS communication. The certificate information confirms that the certificate was successfully created and is valid for secure communications.



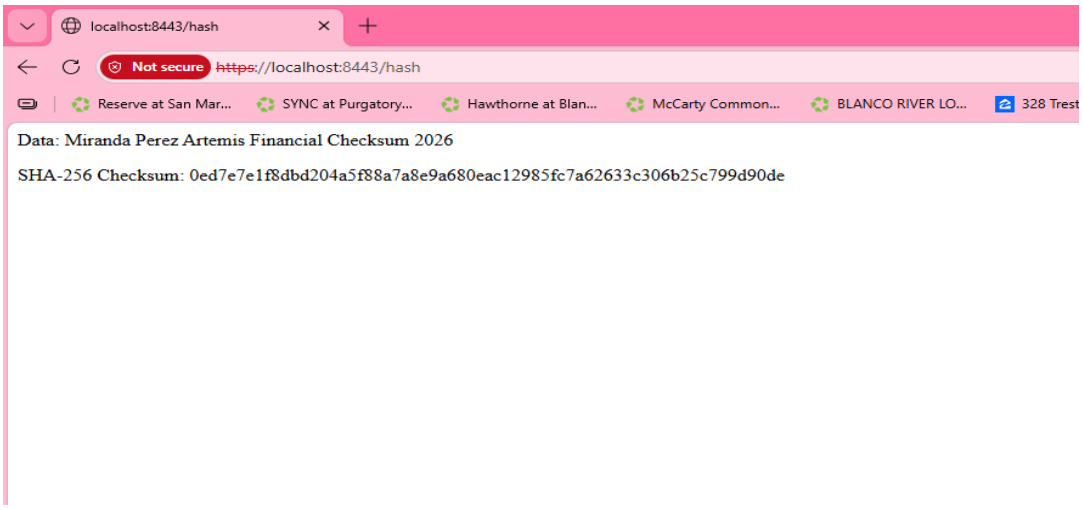


3. Deploy Cipher

Insert a screenshot below of the checksum verification.

The application was refactored to implement the SHA-256 cryptographic hash algorithm. A unique data string containing my name was processed to generate a checksum, which can be used to verify data

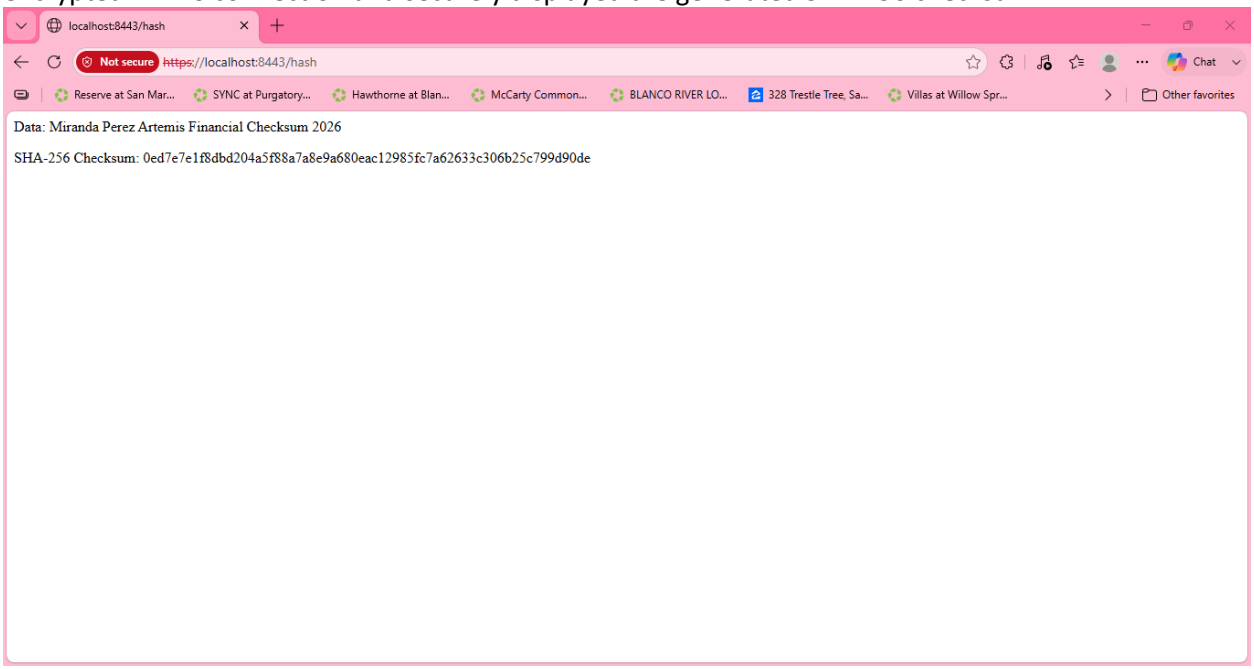
integrity during transmission. The successful checksum output demonstrates that the hash function was correctly implemented and functioning as expected.



4. Secure Communications

Insert a screenshot below of the web browser that shows a secure webpage.

The application was configured to use HTTPS on port 8443 using a self-signed SSL certificate. Secure communication was verified by successfully accessing the application through https://localhost:8443/hash. Although the browser displays a warning because the certificate is self-signed and not issued by a trusted Certificate Authority, the application successfully established an encrypted HTTPS connection and securely displayed the generated SHA-256 checksum.



5. Secondary Testing

Insert screenshots below of the refactored code executed without errors and the dependency-check report.

OWASP Dependency Check was executed to perform secondary static testing on the refactored application. The tool analyzed 49 dependencies and generated a vulnerability assessment report. Review of the report confirmed that the identified vulnerabilities were associated with third-party framework dependencies included in the original project. The SHA-256 checksum implementation and HTTPS configuration added during refactoring did not introduce additional security vulnerabilities.

```
<terminated> ssl-server_student (3) [Maven Build] C:\Users\miran\.p2\pool\plugins\org.eclipse.justj.openjdk

See the dependency-check report for more details.


[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  01:03 min
[INFO] Finished at: 2026-06-19T21:42:45-05:00
[INFO] ------------------------------------------------------------------------
```

DEPENDENCY-CHECK

Dependency-Check is an open source tool performing a best effort analysis of 3rd party dependencies; false positives and false negatives may exist in the analysis performed by the tool. Use of the tool and the reporting provided with regard to the analysis or its use. Any use of the tool and the reporting provided is at the user's risk. In no event shall the copyright holder or OWASP be held liable for any damages whatsoever arising out of or in connection

How to read the report | Suppressing false positives | Getting Help: github issues

Sponsor

Project: ssl-server

com.snhu:ssl-server:0.0.1-SNAPSHOT

Scan Information (show all):

- *dependency-check version*: 12.1.0
- *Report Generated On*: Fri, 19 Jun 2026 21:42:41 -0500
- *Dependencies Scanned*: 49 (31 unique)
- *Vulnerable Dependencies*: 15
- *Vulnerabilities Found*: 218
- *Vulnerabilities Suppressed*: 0
- ...

Summary

Display: Showing Vulnerable Dependencies (click to show all)

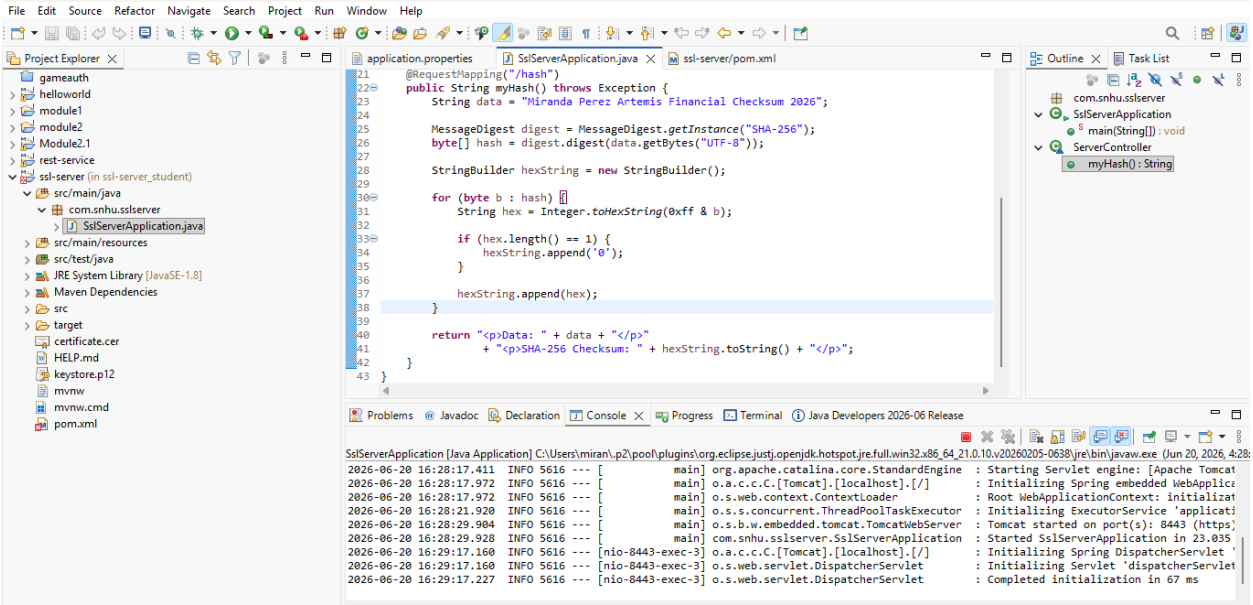
Dependency	Vulnerability IDs	Package

6. Functional Testing

Insert a screenshot below of the refactored code executed without errors.

Functional testing was performed by manually executing and reviewing the refactored application to identify any syntactical, logical, and security issues. The application compiled and executed successfully without errors, as shown in the console output. Testing confirmed that the application established a

secure HTTPS connection and correctly generated a SHA-256 checksum, demonstrating that the refactored code functioned as intended.



7. Summary

The Artemis Financial application was refactored to improve software security by implementing multiple layers of protection. Security improvements included generating and deploying a self-signed SSL certificate, configuring HTTPS communication, and implementing a SHA-256 checksum to verify data integrity. The application was tested using OWASP Dependency Check to identify vulnerabilities within third-party dependencies and to confirm that the refactored code did not introduce additional security risks. These enhancements addressed several stages of the vulnerability assessment process, including secure communications, data integrity verification, vulnerability assessment, and secure software configuration.

8. Industry Standard Best Practices

Industry-standard secure coding best practices were followed throughout the refactoring process to maintain and improve the application's security posture. HTTPS was implemented to encrypt communications between the client and server, while SHA-256 hashing was used to verify the integrity of transmitted data. Dependency Check was used to identify known vulnerabilities in third-party libraries and dependencies. Applying these best practices helps organizations reduce security risks, protect sensitive customer information, maintain customer trust, and support compliance with industry security standards. Consistently following secure coding practices contributes to the overall reliability and long-term success of the software application.